

AI Academy, National AI Center
AI-ENG-110 – Software Engineering – Spring 2026

Topic 4 – Async Research Assistant

Team1

Nurana Aliyeva • Elshan Shukurov • Ali Hasanli • Gasham Huseynli

Final Project Defense – 19 September 2026

`github.com/Nurana100/team1-ai-eng-research-assistant • tag v1.0-final`

The problem in one sentence

What the system does

A user asks one natural-language research question; we fan out to Wikipedia, arXiv and web search *at the same time* and return a single synthesized answer in which every claim carries a citation index pointing at a real URL.

Inputs

- ▶ a question, ≤ 500 characters
- ▶ optional source filter
wiki, arxiv, web
- ▶ .env: provider + keys

Outputs

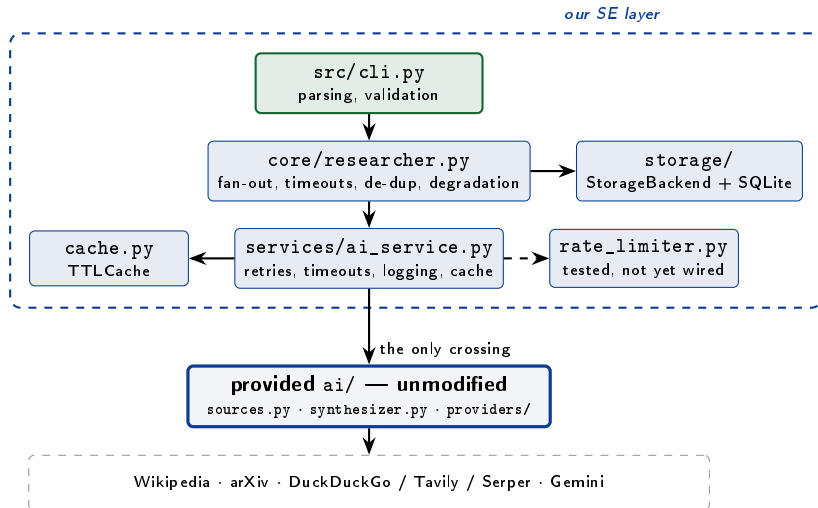
- ▶ cited prose answer + resolved source list
- ▶ one row in SQLite (question, answer, sources, UTC timestamp)

Entry point: one CLI — `python -m src.cli ask "<question>"`

Stack: Gemini gemini-3.6-flash for synthesis, DuckDuckGo for web search (no key needed)

Not built, deliberately: no HTTP API, no cross-question batch mode.

Architecture — one arrow crosses the ai/ boundary



Why it matters: retries, timeouts, logging and caching are cross-cutting — one file to change to add another. Swapping Gemini → Anthropic → OpenAI changes **zero source files**, two lines in `.env`.

Three design decisions we can defend

1. **asyncio, not threads.**

The provided `ai/` fetchers are already coroutines over one shared `httpx.AsyncClient`; a thread pool would need an event loop per thread and throw away the connection pool. Rejected: `ThreadPoolExecutor` (per-source timeouts are awkward — a blocked thread cannot be cancelled), `ProcessPoolExecutor` (no CPU work).

2. **One service class as the only seam over `ai/`.**

Every `ai.*` call goes through `AIService`, so retry policy, timeout, structured log and cache live in exactly one file. Trade-off accepted: that file is a chokepoint, and its coverage looks low because the uncovered lines are the provided functions themselves.

3. **StorageBackend as an ABC, not a Protocol.**

We want the failure at construction time: a half-finished `PostgresStorage` must refuse to instantiate, not explode later at the call site. We assert exactly that in `test_abstract_backend_cannot_be_instantiated`.

Concurrency: the benchmark

Live measurement: 3 sources (Wikipedia, arXiv, DuckDuckGo), 3 runs.

Run	Sequential	Concurrent	Speedup
1	1.29 s	0.71 s	1.82×
2	1.12 s	0.34 s	3.29×
3	1.16 s	0.65 s	1.78×
Mean	1.19 s	0.57 s	2.10×

Ceiling $\approx 2.7\times$; **measured mean** $2.10\times$. For a fan-out the limit is not N but $\sum t_i / \max t_i$ — total serial time over the slowest leg (1.20/0.45 s). Run 2 exceeded the ceiling, which we treat as a warm-connection artifact of our harness (sequential pass runs first), not as a result.

The bottleneck is width, not our code. Three sources is not a wide fan-out; batching questions into one gather is the change that would move the curve. Next step for the harness: interleave the two modes and discard a warm-up round.

Robustness: a failure we did not plan

Scenario. During an earlier benchmark run, Wikipedia started answering 403 Forbidden to *every* request — about 30 requests in ten seconds.

What the system did

- ▶ `gather(..., return_exceptions=True)` kept the exception out of the caller
- ▶ `researcher.py` logged WARNING and skipped the dead source
- ▶ User got a normal, correctly-cited **arXiv-only** answer — no traceback

What we expected vs. found

- ▶ Expected the documented missing-User-Agent 403
- ▶ Replayed the *same* header 0.5 s apart → 200 every time
- ▶ Real cause: **our own request rate**

What the experiment established

The header is necessary but not sufficient — what Wikimedia rejected was one identity making thirty rapid requests. A rate condition, not an authentication one: the fix is a per-source minimum interval (our limiter exists, not yet wired in), not a header.

CLI, cache and Docker

CLI — `src/cli.py`

- ▶ `python -m src.cli ask`
"`<question>`" starts the async pipeline
- ▶ options: `--sources` to pick sources,
`--no-cache` to skip the cache

TTL cache — `services/cache.py`

- ▶ TTL = time to live: a repeated question is answered from memory, with no new API call
- ▶ key = source name + lower-cased, trimmed question, so wiki and arXiv stay separate
- ▶ wired into AIService for wiki, arXiv and web

Tests

- ▶ `tests/test_cache.py`: store and read, expiry, normalisation, per-source keys
- ▶ `tests/test_researcher.py`: success, partial failure, total failure, 3 sources in parallel

Docker and demo runner

- ▶ Dockerfile: same Python environment, runs as a non-root user
- ▶ `scripts/run_demo.py`: loads the 5 prepared questions and runs the pipeline for each

What we will run

One live question end to end, then the same question with arXiv excluded to show a source-filtered run, then an empty question to show a clean validation error.

```
$ python -m src.cli ask "Quantum Entanglement"  
$ python -m src.cli ask "Quantum Entanglement" --sources wiki,web  
$ python -m src.cli ask ""
```

- ▶ Every sentence in the answer carries a [n] index that resolves to a printed URL — no uncited prose.
- ▶ A dead source gives a shorter source list, not a crash; bad input gives one Error: line, not a traceback.

Backup: `artefacts/demo_run_all.txt` holds a captured five-question run if the network or the LLM quota fails.

Offline tests and SQLite storage

Coverage & shape

- ▶ Coverage of src/: **76%** (required $\geq 60\%$)
- ▶ Provided ai/ smoke tests: **16/16 passing**, ai/ unmodified
- ▶ **51 tests**, 35 ours + 16 provided, in about 5 s
- ▶ **Fully offline, enforced:** MockTransport raises on any outbound request; an autouse fixture deletes all five provider keys

Three worth calling out

- ▶ **Happy path:** the stub delegates to the *real* synthesize, so citation numbering is genuine
- ▶ **Concurrency:** one source raises, the other two still come back
- ▶ **Error path:** storage tests caught a leaked SQLite connection and an unchecked lastrowid

SQLite storage — storage/repository.py

StorageBackend ABC (save, get by ID, list recent) → SQLiteStorage: question, answer, sources as JSON, UTC time. It creates missing folders, closes every connection and checks the row id; database files are git-ignored. Test stubs live in tests/conftest.py.

Config, validation and integration

Configuration — `src/config.py`

- ▶ Pydantic Settings: typed provider, model, keys, log level, TTL, max parallel
- ▶ from environment or local `.env`; no keys in code

Input validation — CLI

- ▶ rejects empty or 500+ character questions, unknown or empty source lists

Rate limiter —

`services/rate_limiter.py`

- ▶ per-source minimum interval; tested, *not yet wired* into AIService

Final state: storage wiring merged with the latest main, conflicts resolved, suite re-run — **51 tests pass.**

Checks

- ▶ `tests/test_cli.py` and `tests/test_config.py`
- ▶ mypy report + `docs/security_check.md`: no TODOs, no hard-coded keys

Storage integration

- ▶ CLI creates `SQLiteStorage`, passes it to `run_research_pipeline`
- ▶ saves question, answer, sources, UTC time; a test reads it back

Limitations & what we would do next

Where the system is bounded today

- ▶ **Wikipedia's opensearch does not answer questions.** “What is photosynthesis and what are its main stages?” → 0 hits; “photosynthesis” → 3. The demo maps each known question to a keyword; a product needs query rewriting.
- ▶ **The fan-out is only three wide.** The ceiling is $\sum t_i / \max t_i$, so it stays below 3× until batch mode widens it — what MAX_PARALLEL bounds.
- ▶ **The cache is per-process.** Right for a long-running caller; a one-shot CLI run starts cold. Persisting it into the SQLite file we already write closes that.
- ▶ **No provider failover, no answer-level validation.** A fluent answer that misreads its own citation goes undetected — a research problem, not a weekend one.
- ▶ **SQLite is single-writer.** The StorageBackend ABC exists so Postgres can be dropped in without touching a caller.

Given another week, in priority order

1. Batch mode with a real semaphore bound — a workload wide enough to show the design.
2. Wire in the rate limiter; query rewriting; persisted cache; CI (pytest + mypy).

Team contributions

Member	Primary ownership	Commits	Share
Nurana Aliyeva	CLI, pipeline orchestration, AIService, docs	21	29.2%
Elshan Shukurov	Offline test infrastructure, storage layer, config	19	26.4%
Ali Hasanli	first async CLI, TTLCache + wiring, Dockerfile, researcher tests	19	26.4%
Gasham Huseynli	Rate limiter, CLI validation + tests, security audit	13	18.1%
Total		72	100%

We can all answer

Every change reached main through a reviewed pull request. Pick a file — we will walk through it.

AI tool use: Claude drafted the test stubs, the tenacity decorator shape, the storage ABC and the benchmark script; we narrowed the retryable exception set, added `reraise=True`, and fixed two real defects in the storage draft. Per-module disclosure in report §9.

Questions?

Team1 – Topic 4 – Async Research Assistant

`github.com/Nurana100/team1-ai-eng-research-assistant`

`tag v1.0-final`

“We can walk through any file in the repo.”